

React Native Developer Assessment Brief

Candidate Assessment • Fresher / Junior React Native + MERN Stack Developer • Completion Window:
48 Hours

1. Objective

This assessment is designed to evaluate your practical React Native development skills, project structuring approach, problem-solving ability, and code quality. We are assessing not only whether the app works, but also how you organize the codebase, handle forms and local persistence, manage application state, and deliver a complete mobile application within a defined timeline.

2. Project Assignment

Build an **Expense Tracker Mobile App** using React Native. The application should allow users to record income and expense entries, categorize transactions, view transaction history, and see a simple financial summary. The focus of this assessment is on clean implementation, good app structure, local data persistence, and a usable mobile interface.

3. Core Functional Requirements

3.1 Splash Screen

Create a simple splash screen that displays the app name/logo or placeholder branding. It should automatically navigate to the main app after approximately 2 seconds.

3.2 Dashboard / Home Screen

The main screen should provide a quick overview of the user's finances. At minimum, it should display:

- Total balance
- Total income
- Total expenses
- A recent transactions list

The values should update automatically based on the entries stored in the app.

3.3 Add Transaction Screen

Create a form to add a new transaction. The form must include:

- Title / Transaction Name (required)
- Amount (required, numeric)
- Type: Income or Expense (required)
- Category (required)
- Date (required)
- Notes / Description (optional)

Suggested categories may include:

- Salary
- Food
- Travel
- Shopping
- Bills
- Entertainment
- Other

Validation should be added for all required fields.

3.4 Transaction List Screen

Create a screen that displays all transactions in a clean list. Each transaction item should show at least:

- Title
- Amount
- Type
- Category
- Date

Users should be able to:

- View the full transaction details
- Edit a transaction
- Delete a transaction

3.5 Transaction Details Screen

When a user taps a transaction, open a details screen showing all related information, including title, amount, type, category, date, and notes. Provide actions to edit or delete the transaction from this screen.

3.6 Search and Filter

Add search and filter functionality so users can easily find transactions.

Minimum expected functionality:

- Search transactions by title
- Filter by type (Income / Expense)
- Filter by category

Optional enhancements:

- Filter by date range
- Sort by latest / oldest / highest amount

3.7 Summary Screen

Create a separate summary screen to display financial insights based on the stored data. At minimum, show:

- Total income
- Total expenses
- Current balance
- Number of transactions

You may also add a simple breakdown by category or a visual chart, but that is optional.

3.8 Local Data Persistence

All transaction data must be stored locally on the device using **AsyncStorage** or another appropriate local persistence method. Data should remain available after app restart.

4. Technical Requirements

The project should be built using React Native. You may use either Expo or React Native CLI, though Expo is preferred for faster setup unless there is a specific reason not to use it. Use React Navigation for screen navigation. You may use JavaScript or TypeScript, with TypeScript preferred. The project should be reasonably structured into reusable components, screens, navigation, utilities, and storage or service layers where applicable.

5. Suggested Minimum Screens

At minimum, the app should include:

- Splash Screen
- Dashboard / Home Screen
- Add / Edit Transaction Screen
- Transaction List Screen
- Transaction Details Screen
- Summary Screen

6. UI / UX Expectations

The UI does not need to be highly polished, but it should be clean, readable, and usable on a mobile device. We will evaluate spacing, consistency, readability, form usability, and overall layout quality. Empty states, validation feedback, and smooth navigation are appreciated.

7. Code Quality Expectations

Please keep the code clean and maintainable. We will look for:

- Meaningful naming conventions
- Reusable components where appropriate
- Logical folder structure
- Avoidance of unnecessary duplication
- Proper use of React hooks and state
- Clear separation of UI, logic, and storage concerns where possible

8. Deliverables

Please submit the following:

- a) Git repository link containing the complete source code
- b) APK file or downloadable APK link
- c) README.md with:
 - Project setup instructions
 - Steps to run the app locally
 - Tech stack / libraries used
 - Assumptions or limitations
 - Notes on any incomplete items
 - APK build instructions or APK location

9. Submission Timeline

This assessment must be completed and submitted within **48 hours** from the time it is assigned.

10. Evaluation Criteria

Your submission will be evaluated on:

- App functionality and correctness
- React Native fundamentals
- Form handling and validation
- Local storage and state management
- Code quality and maintainability
- App structure, UI consistency, and delivery quality
- APK generation and repository completeness

11. Optional Bonus Points

The following are optional and may strengthen your submission:

- Use TypeScript
- Add date range filters
- Add category-wise expense summary
- Add a simple bar/pie chart for expense breakdown
- Add dark mode support
- Add reusable custom form components
- Add better empty and error states

12. Important Notes

The objective of this assessment is not to build a production-grade application, but to demonstrate your approach to building a small, well-structured React Native app. Focus on correctness, clarity, maintainability, and a smooth user experience.